

Parameters and the Automatic Dialog Builder

coreBuilder Application Note

Ronnie Craig, Synopsys Design Reuse Group

July 2000

Abstract

coreBuilder uses parameters to represent elements of data which are to be user-configurable. This Application Note is primarily concerned with parameters which are derived from the HDL source code. The coreBuilder tool provides the facility to generate a configuration dialog to allow the user to view and edit the HDL parameters.

Introduction

One of the key features of coreBuilder is the ability to capture a set of HDL parameters associated with a given core, and make them directly accessible and configurable by the core user. This is essentially a two stage process - first of all, the appropriate HDL parameters are suitably defined, and secondly, these definitions are parsed by coreBuilder and an appropriate configuration dialog is created.

Specification of HDL

coreBuilder needs to be able analyze the HDL of a core design so that it can build up a model of the design that both the core developer (in coreBuilder) and the core integrator (in coreConsultant) can interact with. This model needs to contain the top-level parameters and user modifiable constants, and any other parameters or constants that are used as part of the definition of the structural netlist on or above the implementation frontier.

Intent Specification

Configuration intent indicates which parameters/constants can be seen by the core integrator and which can be modified. More importantly, this intent defines the valid configuration space, preventing the core integrator from creating an invalid configuration.

This intent information is also used to control the look and feel of the graphical user interface used for interaction with the configuration data.

Intent information can generally be specified either as annotations in the HDL, as commands in an intent command file, or interactively via the spreadsheets available within coreBuilder. When the same intent is specified in more than one way, the precedence (from lowest to highest) is HDL annotation, intent command file command, and then interactive specification.

Given the fact that configuration intent enhances the code it is in and is fairly stable, it is recommended that all configuration intent be specified via HDL annotations.

Annotation Syntax

Annotation pragmas are associated with design objects based on where they are located in the HDL file. An annotation is stored until it can be "attached" to an item in the HDL. The annotation will attach to the first identifier (language independent) or scope closing token (language dependent) encountered.

Several different types of pragmas are available for use in HDL annotations. Each starts with the string "reuse-pragma", followed by an annotation directive. The valid annotations are:

```
reuse-pragma attr [if(< expr>)] < attr> < value>

reuse-pragma beginAttr [if(< expr>)] < attr> <value>
reuse-pragma endAttr
```

The 'attr' pragma is the one that will be used in almost all cases. The standard form of this annotation will cause the intent attribute <attr> to be set to <value> on the design object associated with the annotation. The filtered version (when <expr> is specified) applies to the set of HDL elements in the current HDL structural section.

If the attribute value requires a large amount of space to represent, the beginAttr/endAttr annotation can be used to divide the annotation specification up over multiple lines. The following example shows where annotations should be placed to cause them to be associated with different types of design objects.

```
entity FSM is
-- reuse-pragma attr EnumValues 8 16 32
generic (bus_width: unsigned := 8);
port ( into : in bit_vector(bus_width-1 downto 0);
      outof : out bit_vector(bus_width-1 downto 0)
end FSM;
```

In the above example the EnumValues attribute is set on the bus_width generic. Annotations can be applied to user modifiable constants in a similar manner.

Intent Attribute Definitions

This section documents the intent attributes. The lists are divided into tables based on how they are grouped for interactive specification.

The keys used in the Category column are defined as follows:

M mandatory,

R recommended

Attribute	Type	Default	Category	Description
Label	string	<parameter_name>	R	The string shown for a parameter in the parameter entry dialog.

Attribute	Type	Default	Category	Description
Description	string		R	A description of the parameter (i.e. 'indicates the width of the data bus')
MinValue	matches param	<from HDL if possible>	M**	The minimum legal value for a numeric parameter.
MaxValue	matches param	<from HDL if possible>	M**	The maximum legal value for a numeric parameter.
DefaultValue	matches param	<from HDL if defined>		Default value for the parameter.
Value	matches param	<from HDL if defined>		Actual value for a parameter.
GroupName	string		R	The name of the parameter dialog tab that this parameter should be displayed in.
ParameterInfo	string			Embedded Tcl script which contains structural information for the parameter configuration dialog.
ReadOnlyParam	boolean	0		Indicates that the core integrator can not modify the parameter (including its value) in any way.
SymbolicNames	string			The list of symbolic names to use for the full range of valid values for a parameter (i.e. 'one two three' for an integer parameter or 'off on' for a boolean parameter).

Attribute	Type	Default	Category	Description
EnumValues	string	<from HDL>		The list of valid values for a parameter. This is typically extracted from the HDL unless only a subset of the HDL defined values are actually legal.
ArrayStart	integer	<from HDL if defined>		The index of the first element of an array parameter.
ArrayEnd	integer	<from HDL if defined>		The index of the last element of an array parameter.
ArrayFieldSize	integer	<from HDL if possible>		The size of an array parameter.
CheckExpr	boolean	1		TCL expression used to verify the validity of a parameter value. Can be set up to apply under specific conditions (see manual).
CheckExprMessage	string			Message to be reported when CheckExpr fails
CrossParamGroup	string			If set on a parameter, indicates that the parameter is part of a multiple parameter mutual exclusivity check.
ParameterCheckCmd	TCL script			Defines a script which is executed when a parameter check is executed.

Attribute	Type	Default	Category	Description
RadioCheckedVal	string			Indicates that the parameter is to be set using a radio button, and provides the value for the parameter when the radio button is selected.
AltValue	string			Alternate value for the parameter. See the manual for a description of how alternate valued parameters can be used.
Sequence	integer			Determines the order in which the parameters appear in the configuration dialog.

****** these parameters should not be specified by the developer unless they are to be more restrictive than the existing type declaration.

Parameter Checking

To ensure that the core is being configured legally and correctly, the core developer can also use a number of checking attributes when specifying configuration intent. These 'checks' will report default or developer-specified messages when configuration checks fail. For example, given the following intent specification -

```
-- reuse-pragma attr MinValue 0
-- reuse-pragma attr Max Value 16
CONSTANT myconstant : INTEGER := 7;
```

If, in the configuration dialog, the user specifies a value for `myconstant` which is outside the range 0-16, an error message will be reported.

Cross-dependencies can also be setup between parameters, such that the legal value(s) for one parameter is/are dependent on the value(s) of one or more other parameters. For example -

```
set_parameter_attribute A MinValue (=B)
```

The attribute definition above indicates that the minimum value of the parameter A must be greater than or equal to the value of the parameter B.

In addition to the standard attribute-related checks specified above (such as `MinValue` and `MaxValue`), it is also possible for the core developer to specify a set of more detailed parameter checks. Checks can either be performed locally to a particular parameter, or globally across a number of parameters.

Expression Checking

The `CheckExpr` parameter attribute enables you to perform automatic error checking on user-specified values for activity parameters.

`CheckExpr` is a case subscripted attribute in which the subscript is a boolean expression that `coreBuilder` evaluates. If the expression is true, then `coreBuilder` also evaluates the value of the attribute as a boolean expression. If the value of that expression is false, then `coreBuilder` returns an error message that describes what required conditions were not met.

In addition, the `CheckExprMessage` allows the developer to specify the exact message string associated with each check. The usage of these expressions is shown by the example below -

```
-- reuse-pragma attr CheckExpr @mem_end[@mem_end>1] >
                                @mem_start
-- reuse-pragma attr CheckExprMessage "End address must be
                                greater than start address"
CONSTANT mem_end : INTEGER := 512;
```

Parameter Uniqueness & Global Parameter Checking

In addition to checking the configuration of individual parameters, cross-parameter and global checks can be executed. In this way, the developer can specify certain relationships between parameters and checks which should be equally applied to all parameters.

Cross Parameter Checking

An attribute is available called `CrossParamGroup`, which simplifies cross-parameter uniqueness checks. By adding this attribute to selected parameters, they are effectively added to a unique error-checking group. An error will then be reported if the set of 'grouped' parameters are not unique.

This is demonstrated in the example below -

```
-- reuse-pragma attr CrossParamGroup interrupt
CONSTANT int1_priority : INTEGER := 1;
-- reuse-pragma attr CrossParamGroup interrupt
CONSTANT int2_priority : INTEGER := 2;
-- reuse-pragma attr CrossParamGroup interrupt
CONSTANT int3_priority : INTEGER := 3;
```

In this example, the use of `CrossParamGroup` forces each parameter to have a unique value.

Global Parameter checking

It is possible to execute global checks across all configuration parameters using the `ParameterCheckCmd` attribute. This attribute specifies a Tcl procedure which is called to execute a check when a parameter check is performed (i.e. when the user clicks 'Check' or 'OK' in the configuration dialog).

The `ParameterCheckCmd` can be used as follows -

```
coreBuilder>set_parameter_attribute <design_name>  
ParameterCheckCmd {<tcl proc. name>}
```

Alternatively, the global parameter check can be inserted in the HDL file before the top level module/entity declaration.

GUI Hints for Parameters

Once the basic semantics of the parameter are defined additional attributes can be supplied to give hints to the GUI builder on how to best generate a user interface for the parameter. These GUI hints are given below:

Table 1 *Parameter GUI ‘hint’ attributes*

Attribute Name	Type	read/write	Default	Description
Sequence	Integer	r/w		Specifies the order in which the parameters should appear in the dialog. If the parameters have been grouped (using GroupName) then the smallest sequence number of the parameters in that group determines the order of the groups.
GroupName	String	r/w		Defines a group of related parameters. Parameters with the same group name will be grouped into separate tabs in a property sheet. The tab on the sheet will be given the GroupName. The sequence of the parameters determines the sequence of the GroupNames and thus the order of the tabs in the property sheet.
Label	String	r/w	Parameter Name	Specifies the label to be used on the entry in the dialog.
HelpURL	String	r/w		Specifies the path to the document describing the parameter in the IP documentation. Only the part of the path from the core installation should be specified.
Enabled	Boolean	r/w	True	Specifies that the parameter should be enabled for user input. This is a case-subscripted attribute, see the section below for details.
Visible	Boolean	r/w	True	If set to TRUE then the GUI builder will generate controls for this parameter. If FALSE the parameter will not appear in the dialogs generated.

Attribute Name	Type	read/write	Default	Description
AltValue[valueLabel]	String	r/w		Label to be used on checkbox or radio button used to enable setting the value of this parameter to its non-alternate value.
AltValue[alternateLabel]	String	r/w		Label to be used on the radio button for the case where the AltValue is used.
ValueType	Enum	r/w	string	Provides a hint to the builder indicating that a string is actually a special kind of string. The builder can build a custom control for editing this type of a value (note: this directive does not apply to HDL parameters) • Builds an edit with a browse button hooked to a file dialog. • directory Builds and edit with a browse button hooked to a directory dialog. • fileList Builds an editable list dialog with the browser hooked to a file dialog. • directoryList Builds an editable list dialog with the browser hooked to a directory dialog.

The Enabled attribute is a case-subscripted attribute, which means that both its subscripts and their values are expressions. When evaluating a case subscripted attribute we first evaluate all of the subscripts. For each of the subscript expressions which evaluate to true (non-0) we then evaluate the values of the attribute for those subscripts and combine the values with a boolean operator. In the case of the enabled attribute, the values are combined with an '&&' (boolean AND) operation.

This sounds much more complicated than it really is. A short example should help to clear things up. For a case where a parameter B only should be set if parameter A was set to 5, the following attribute could be used

```
set_parameter_attribute B {Enabled [@A == 5]} 1
```

Also note, that the default subscript for the Enabled attribute is 1, so it is also valid to simply set the Enabled attribute to a value such as

```
set_parameter_attribute B Enabled 1
```

Configuration Dialog Layout

CoreBuilder provides a facility to automatically generate a dialog or property sheet for a list of parameters. This facility is used to build the dialogs used to specify the configuration of a design, as well as other places internal to the tool to build dialogs for activities and for entering formulae.

The facility attempts to create a GUI that makes it easy for the user to correctly specify legal values of the parameters. The current set of heuristics for building the dialogs will be detailed below. These may change over time as better heuristics come to light, but will give you a general feel for how the dialogs are generated.

Building controls for Parameters

The rules for how the controls for a parameter are built are given below:

Parameters with the 'EnumType' attribute set

- If the enumType is 'boolean' then create a checkbox
- If there are only two entries create radio buttons
- Else create a combo box

Boolean Parameters

- If symbolicNames are specified for the parameter, generate radio buttons
- Else create a check box

String/Integer/Float Parameters

- If there are symbolicNames for the value and the names are not "false", "true"
 - If there are two names then use a radio button
 - Else use a combo box
- If MinValue = 0 and MaxValue = 1 use a check box
- Use an edit field.
 - If the edit field has AltValue[actual] set on it, then create a checkbox or radio buttons to control enabling the value/ selecting the AltValue.
 - If the string value has the ValueType attribute set to a value other than string, build the custom editing controls for the string. The valid values for ValueType and the controls that are built for each value are given in the *Table 1: Parameter GUI 'hint' attributes*.

Note that parameters without DefaultValues specified have an implied value of " allowed for them. Therefore, if you have an integer value with two symbolic names and no default value you will end up with a combo box being generated (so that ", name1, and name2) are the values. However, if that parameter had a DefaultValue then radio buttons would be generated because the " value is not legal.

Parameters with the AltValue[actual] attribute set on them have the above controls augmented by a checkbox or set of radio buttons. Remember that AltValued parameters have an alternate value that is outside of the basic semantics of the rest of the parameter values. This means that the GUI needs to add an additional control which indicates if the parameter is to be set to it's Alternate value, and if it is then the controls generated for setting the value for the normal value range are disabled. If the AltValue[actual] is set and AltValue[altValueLabel] is set then a set of radio buttons will be used, if it is not set then a checkbox will be used.

Grouping parameters into a dialog or property sheet

The set of parameters for a design is first sorted by Sequence number. We then look at the GroupName of the parameters in order to build up a list of the valid parameter Groups. Groups that contain over 15 parameters are automatically split into multiple groups. If only a single group is defined for the parameters then a dialog is built and the controls for the parameters are added to it.

If there are multiple groups then a property sheet will be generated with a tab for each of the groups. The following shows the relationship between a set of reuse-pragmas and the configuration dialog layout.

```

// reuse-pragma beginAttr Description
// The Core Integrator can choose to have a cache memory
// reuse-pragma endAttr
// reuse-pragma attr Sequence 0
// reuse-pragma attr GroupName Cache
// reuse-pragma attr Label Cache Memory Included ?
// reuse-pragma attr SymbolicNames false true
// reuse-pragma startSub [ReplaceConstantParam %subText]
'define USE_CACHE 1'b0

// reuse-pragma beginAttr Description
// Define the (optional) cache size
// NOTE: If USE_CACHE = 0 this parameter is disabled
// reuse-pragma endAttr
// reuse-pragma attr Sequence 1
// reuse-pragma attr GroupName Cache
// reuse-pragma attr Label (Optional) cache size
// reuse-pragma attr Enabled[!@USE_CACHE] 0
// reuse-pragma attr SymbolicNames 8k 16k
// reuse-pragma startSub [ReplaceConstantParam %subText]
'define CACHE_SIZE 1'b1

// reuse-pragma attr Visible 0
// reuse-pragma attr ReadOnlyParam true
/// reuse-pragma startSub [ReplaceConstantParam %subText]
'define CACHE_SIZE_user ('USE_CACHE) ? 1'b1 : 'CACHE_SIZE

```

This is represented by the configuration dialog shown in *Figure 1: Configuration Dialog Example* below. Note that this dialog contains other parameter groups not shown in the example code above.

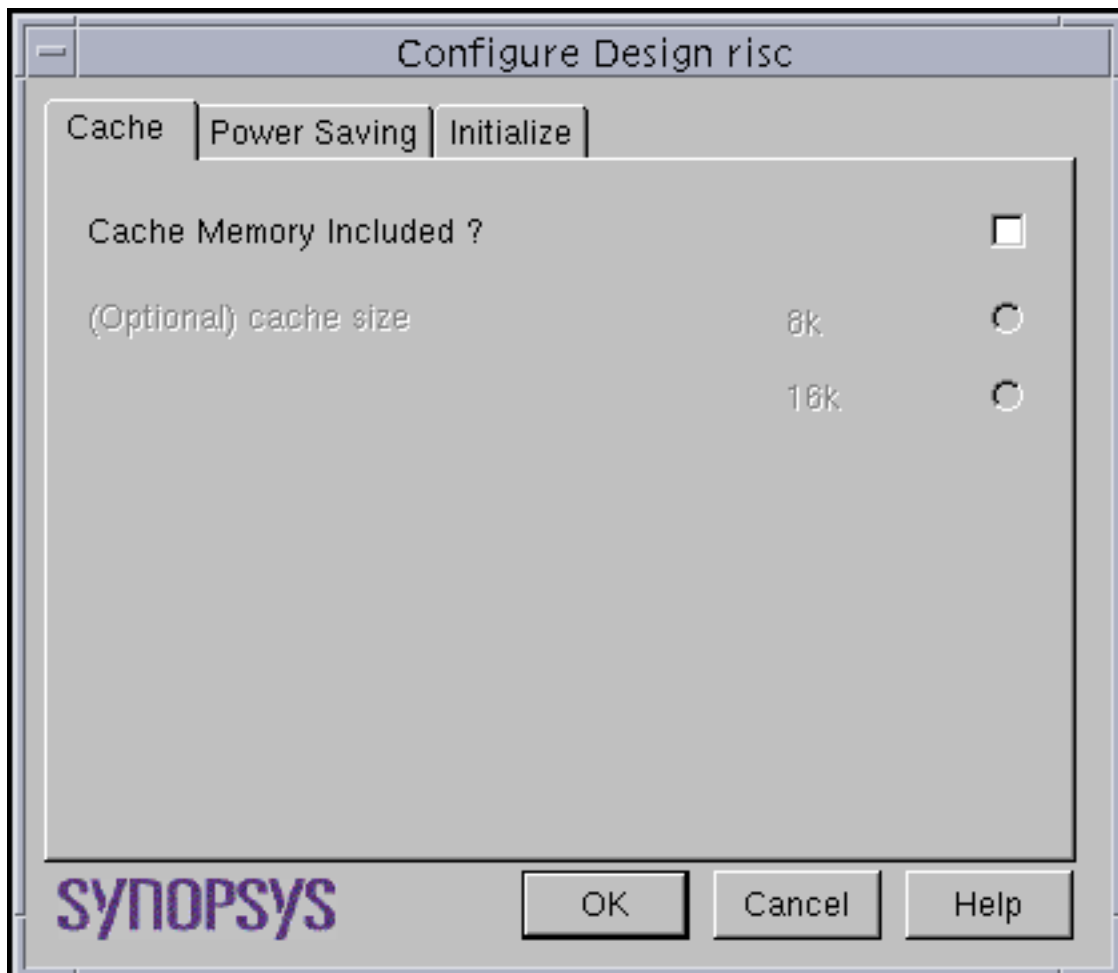


Figure 1 Configuration Dialog Example

To give more control over parameter layout in the dialog, it is also possible to have nested groups of parameters. This can be achieved by using the `ParameterInfo` attribute.

Note: The `ParameterInfo` attribute and associated intent definitions must be inserted before the top-level module/entity declaration as shown in the example below -

```

// reuse-pragma beginAttr ParameterInfo
// {MaxParamCount 20}
// {ContainerType tab}
// {Group "Power Saving"
//   {Label "Power Saving"}
//   {Parameters {ADD_ALU_LATCHES}}}
// }
// {Group Init
//   {Label Initialize}
//   {Parameters {INITIALIZE_REG REG_INIT_VALUE}}
//   {Group Size
//     {Parameters {ADD_WORDLENGTH}}}
//   }
// }
// {Group Cache
//   {Label Cache}
//   {Parameters {USE_CACHE CACHE_SIZE}}}
// }
// reuse-pragma endAttr

```

```
module risc
```

```
...
```

These attributes will give a dialog as shown in *Figure 2: Hierarchical Parameter Grouping*. Note the hierarchical grouping (the ‘Size’ group is arranged hierarchically below the ‘Initialize’ group).

The ParameterInfo attributes can also be specified in a Tcl intent file for this design, if preferred.

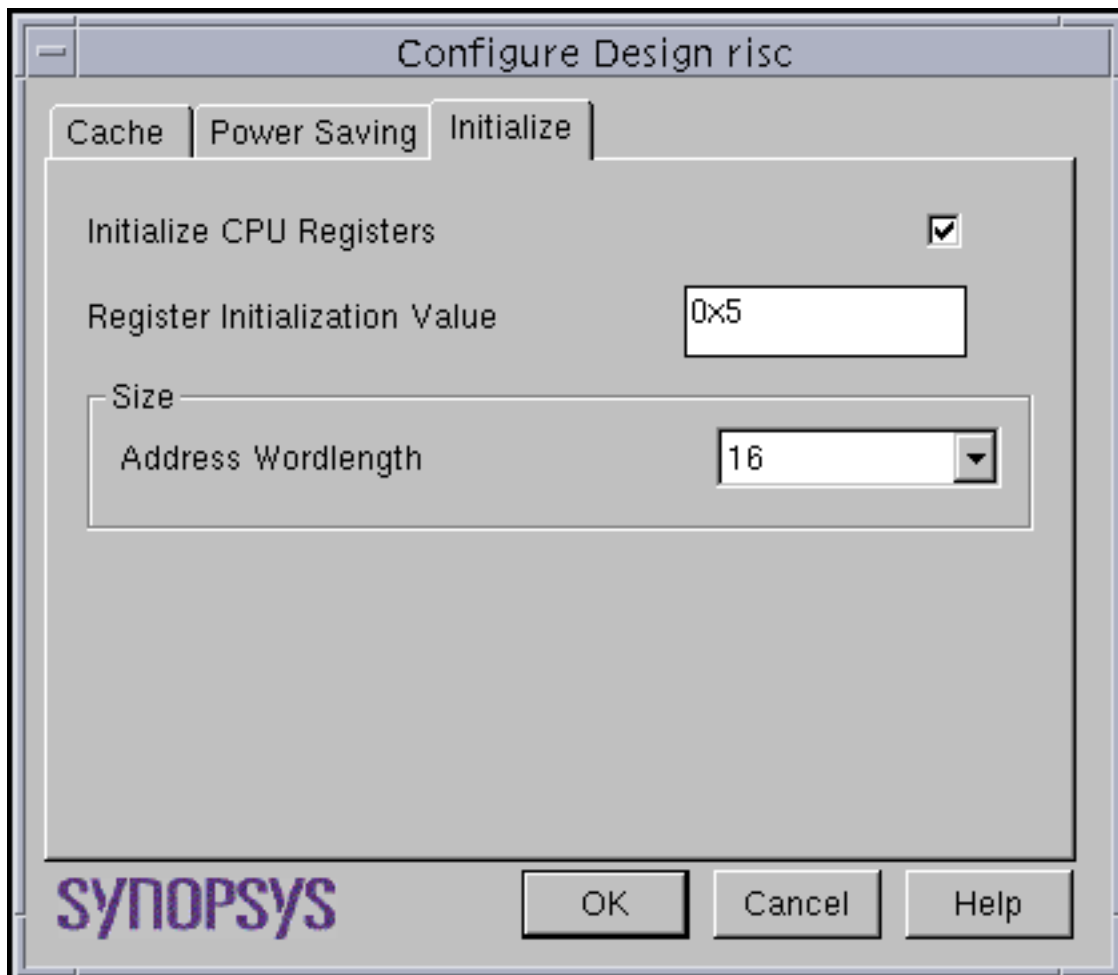


Figure 2 Hierarchical Parameter Grouping

Help

The Help facility is an integral part of the CoreBuilder tool, and the parameter dialogs automatically support both the short and extended help available within the tool.

The short, or 'What's This?' help for a parameter is automatically generated from the Description for the parameter and some of the attributes which define the valid values for the parameter. In addition the Help button on the generated dialog is automatically linked to an auto-generated Html page which lists the 'What's This?' help for each of the parameters in the dialog as well as a link to the HelpURL specified for those parameters.

Appendix 1 Text Substitution

During the core integration process, the HDL files are written out in a specific configuration as defined by the core integrator. This file customization process is enabled by the core developer through the use of text substitution pragmas. These are typically used to enable substitution of constants and parameter values, but can be used to substitute whatever the core developer would like. There are three flavors of substitution available as indicated by the example below:

```
-- replace next line with the results of the TCL command
-- reuse-pragma startSub [<TCL command>] endSub

-- replace next line and the line below it
-- reuse-pragma startSub [<TCL command>]
random line of text

-- replace the keyed block of lines (key is user definable)
-- reuse-pragma startSub MyKey [<TCL command>]
random line of text 1
random line of text 2
-- reuse-pragma endSub MyKey
```

Note that in the example the pragmas are on lines that begin with '--'. This is not part of the pragma specification. The pragmas can appear anywhere on the line, including starting in column 0. They are typically preceded by the comment indicator for the type of file they are in so that the file is valid prior to doing the text substitution. This generic text substitution mechanism is used to control the replacement of constants and parameters in HDL files. For example, the following should be used to enable replacement of a user modifiable constant with the integrator-specified value:

```
-- reuse-pragma startSub [ReplaceConstantParam %subText]
constant Width : integer := 4;
```

Note that this version does a single constant. There is also a block replacement version as shown in the example below:

```
// reuse-pragma startSub CBlk1 [ReplaceConstantParam
                                %subText]
'define Width 4
'define Height 8
// reuse-pragma endSub CBlk1
```

In addition to replacing constants, parameters (generics in VHDL, parameters in Verilog) can also be replaced for the top-level design. The following block substitution functions are available for this purpose: `replace_vhdl_generic_block`, and `replace_verilog_parameter_block`. The VHDL replacement function will work on either a generic map or on the actual generic definition block.

